

Learning Universe

A learner states a goal in one sentence — by text or voice. Pathwise maps the terrain of skills between them and that goal, plans the route over real courses, explains every step with evidence, adapts the moment they prove something — and renders the whole journey as an explorable 3D galaxy that is unique to each learner.

[illegible]

Team

Team Epoch

AJAY KUMAR GARG
ENGINEERING COLLEGE
Ghaziabad, Uttar Pradesh

AD

Abhishek Dubey
FULL STACK DEVELOPER

★ TEAM LEADER

RS

Rohan Sharma
FULL STACK DEVELOPER

PV

Priya Verma
FULL STACK DEVELOPER

Team Epoch is a three-member full-stack team from Ajay Kumar Garg Engineering College. We split the build across the deterministic engines, the AI seams, and the 3D interface — with every member owning features end-to-end, from database schema to WebGL.

01 Problem & Vision

Every learning platform can list courses. Almost none can answer the four questions that actually decide whether someone reaches a career goal: **What exactly am I missing? In what order? Why this resource, right now? And how sure are you?**

Pathwise is built around those four questions. Skills are not a list but a **prerequisite graph** — a directed acyclic graph where Linear Algebra gates Machine Learning, and Machine Learning gates Deep Learning. A learner's knowledge is a position in that terrain; a goal is a destination; a learning path is a route; and progress is movement the system can measure, explain, and re-plan around. Three pillars carry the product: an **AI digital twin** of what the learner actually knows, an **adaptive engine** that re-plans on evidence, and **explainability** deep enough that every number on screen can produce its receipts — all voiced through a coach you can talk to.

The one design rule everything obeys

The deterministic engine decides; the model only writes prose. Every decision with a right answer — prerequisite ordering, gap size, resource ranking, grading, mastery, adaptive branching — is pure Python: no database, no clock, no network, no model. Same inputs, same outputs, unit-testable, auditable after the fact. The LLM works only where natural language is genuinely required, and both edges are fenced: structured outputs are validated against typed contracts before use, and generated prose passes a grounding guard that rejects any number or skill not present in the evidence.

THE MODEL DOES	THE MODEL NEVER DOES
Extract a structured profile from free text or a resume	Decide a proficiency, a gap, or a score
Design a role's skill graph for goals the catalogue lacks	Write a cycle into the DAG (checked, refused)
Judge the best courses from an engine-vetted shortlist	Add a course the engine didn't approve
Rephrase a grounded explanation; write the coach's reply	Invent the facts it explains; choose what data to fetch

The system at a glance



The graph is alive: it ships with a curated seed (12 categories, 56 skills, 83 edges) and **grows at runtime** — when a learner names a role the catalogue cannot plan ("DevOps Engineer"), the model designs that role's required-skill graph, deterministic code materialises it through a cycle-checked API, and an autonomous catalogue pipeline finds real courses for the new skills. A second learner with the same goal converges on the same graph.

02 End-to-End Workflow

One continuous loop — **learn** → **assess** → **improve** → **adapt** → **grow** — built from eighteen cooperating stages.

1	USER INPUT	natural language, voice, or resume upload
2	GOAL INTELLIGENCE	intent, goal type, target role, timeline, constraints
3	CAREER DISCOVERY	"I don't know what I want" → AI interview → ranked careers
4	LEARNER PROFILING	skills, experience, projects, preferences, availability
5	AI SKILL TWIN	what the learner actually knows, updated by evidence
6	TARGET ROLE ANALYSIS	required-skill vector for the goal (grown if unknown)
7	SKILL GAP ENGINE	current vs required, in prerequisite order
8	RECOMMENDATION	semantic retrieval + hybrid scoring + readiness gate
9	PERSONALIZED PATH	phased, prerequisite-aware roadmap with a schedule
10	3D UNIVERSE	the skill graph rendered as the learner's own galaxy
11	EXPLORATION	click a star → why it's on the path, what it unlocks
12	AI COACH	voice + chat, grounded in tools, never inventing
13	LEARN / PRACTICE	real YouTube courses, in-app projects, checkpoints
14	ASSESSMENT	MCQ, coding, conceptual; exact-match deterministic grading
15	SKILL TWIN UPDATE	evidence-weighted proficiency blend, receipts kept
16	ADAPTIVE ENGINE	threshold bands: complete, unlock, skip, remediate
17	UNIVERSE ADAPTATION	stars ignite, paths open, weak areas glow
18	NEXT BEST ACTION	the single next step, with "why this?" attached

Three ways in

Describe the goal

"I want to become an ML engineer, I have 8 hours a week and I already know Python" — one sentence sets the goal, the weekly budget, and the starting skills. Spoken input follows the identical pipeline.

Paste a resume

The same extraction seam reads experience, projects and skills into a structured draft the learner confirms — the model proposes, typed contracts validate, the learner approves.

"Not sure yet"

Career discovery: a stateless AI interview estimates a preference vector over a fixed trait taxonomy; ranking careers from that vector is pure arithmetic, so the same answers always produce the same ranking.

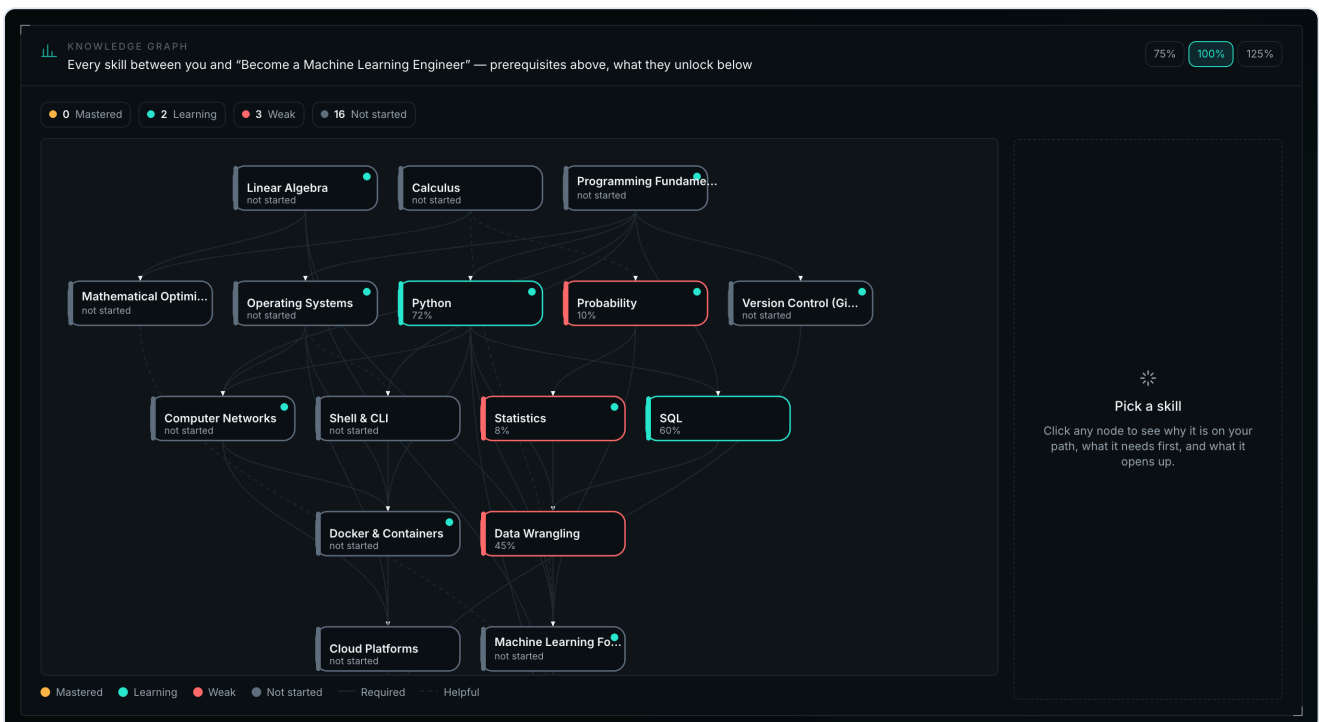
Where each decision lives

STAGE	DETERMINISTIC ENGINE	MODEL CONTRIBUTION
Goal → target skills	Exact catalogue match; curated role fallback	Designs unknown roles' skill graphs (validated, cycle-checked)
Gap analysis	Priority-aware topological sort over the DAG	—
Recommendations	Hybrid scoring, readiness gate, prior-learning suppression	—
Grading & mastery	Exact-match key; fixed threshold bands	Drafts candidate questions (validated before use)
Explanations	Evidence assembly + grounding guard + template floor	Writes the prose, within the evidence

03 The AI Digital Twin — a Living Model of What You Know

At the centre of Pathwise sits a **skill twin**: a per-skill digital model of the learner — proficiency on a 0–1 scale, a confidence weight, the *source* of the evidence, and when the skill was last practised. Every subsystem reads from it; only evidence writes to it.

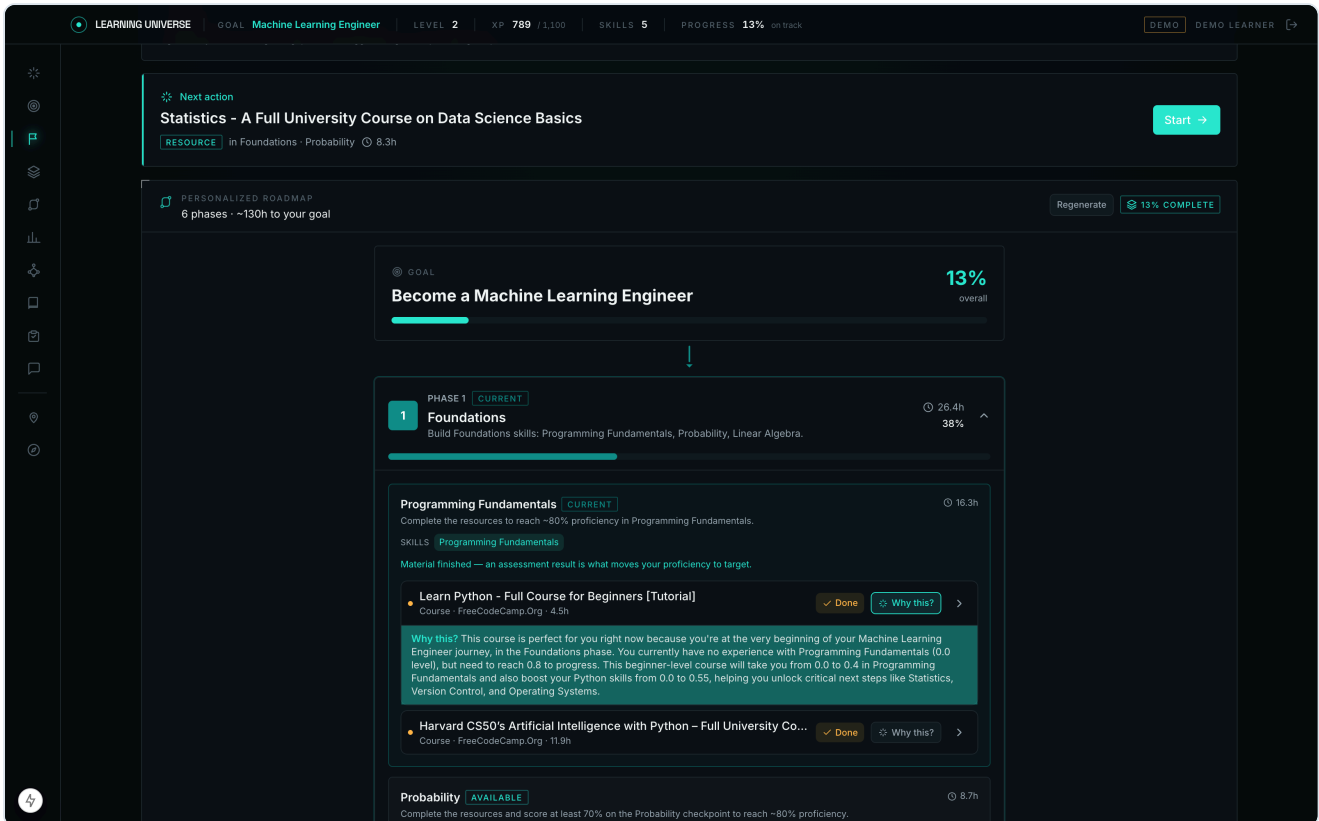
- **Built from three intakes.** A sentence ("I already know Python"), a pasted resume, or the career-discovery interview — each passes the same extraction seam where the model proposes and typed contracts validate before anything is recorded.
- **Evidence has a hierarchy.** A passing remark is not an exam: self-report only ever *fills a blank* — it never overwrites an assessment result, because measured evidence outranks claimed evidence. Every entry carries its evidence source (`self_report` | `completion` | `assessment`) and a confidence that grows with the amount of measurement behind it.
- **Updates are an invertible formula.** Assessment results fold in through a fixed linear blend —
$$\text{new} = 0.6 \cdot \text{old} + 0.4 \cdot \text{score}$$
 — which means any update can be recovered exactly. That is not a detail: it is what makes a reviewer's re-grade *replayable* (recover the pre-submission state, re-apply the corrected score) and every historical delta reportable without storing snapshots.
- **The twin has receipts.** `GET /me/skills/{id}/evidence` returns everything behind a number: the recorded entry, every assessment attempt that measured the skill, and every completed resource that taught it. The UI presents it as "*How do I know this number?*"
- **The twin drives everything visible.** The gap engine subtracts it from the goal's requirements; the readiness gate compares it against each resource's prerequisites; career readiness composes it with assessment and project evidence — and the knowledge graph below literally colours every node from it.



The skill twin, made visible: every node's colour and percentage comes from the twin's current state — teal learning, coral weak, grey unmeasured — using the adaptive engine's own threshold bands, so the view can never disagree with the engine.

04 Explainable AI — Every Number Has Receipts

Most recommenders answer *what*. Pathwise is built to answer *why* — and to prove it isn't making the answer up.



"Why this?" on a roadmap item: the model's prose, generated strictly from the item's persisted rationale and the learner's live records — every percentage traceable.

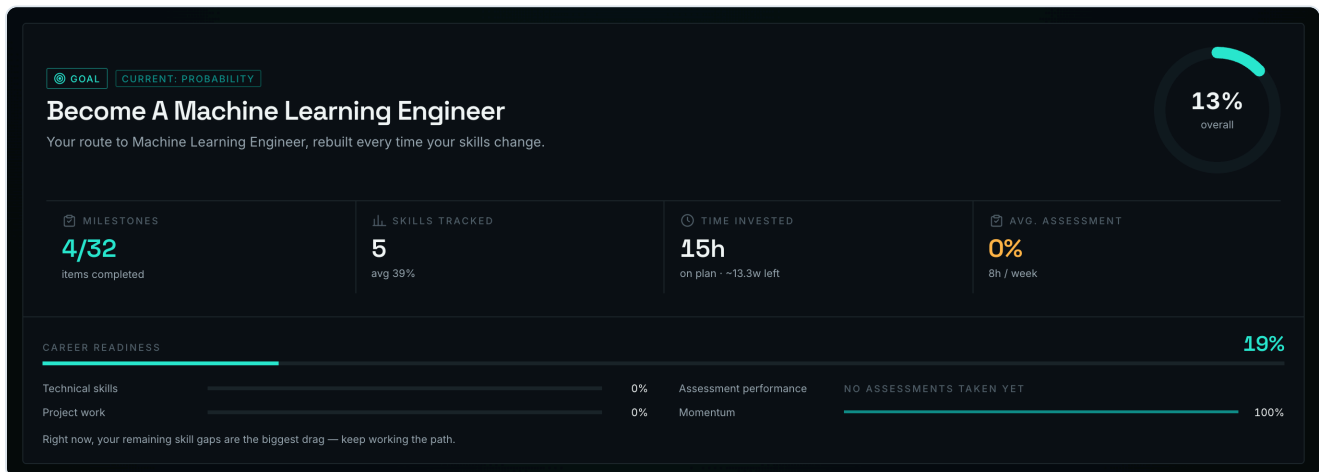
- **"Why this?" on every roadmap item.** Each planned item persists a `rationale_trace` — the skill it serves, current level, required level, the gap, its phase. On demand the backend assembles that evidence with the skill twin's live records; the model writes the prose; a **grounding guard** rejects any output containing a percentage or skill-like phrase absent from the evidence. If the model invents, a deterministic template answers instead. Every response is labelled `source: llm | template` and `grounded: true/false`.
- **Auditable after the fact.** Recommendations and path items persist their full scoring trace, so a past decision can be inspected long after the catalogue has changed.
- **Structure explained from structure.** Why a skill is on the path, what it needs first, and what it unlocks are answered from the prerequisite DAG itself — graph queries, not generated text.
- **The same fencing everywhere.** Extracted profiles and generated assessment questions are validated against typed contracts before use — a malformed or unkeyed question is rejected, never scored. Short answers are never auto-marked; they wait for a human reviewer, whose correction replays the full adaptive pipeline.

05 The Adaptive Learning Engine

The plan is not a PDF — it is a living object that reshapes itself on **evidence**, and only on evidence.

```
profile + goal
  → skill_gap/analyzer      what is missing, in prerequisite order
  → search (pgvector)       candidate resources, semantically retrieved
  → recommendation/scoring  weighted hybrid rank + readiness gate
  → path/generator          phased roadmap with a schedule
  → explanation             rationale, grounded against the evidence
```

- **Prerequisites always come first.** Gaps are not sorted by size — a priority-aware topological sort guarantees a prerequisite precedes whatever depends on it, in every phase of the roadmap.
- **The readiness gate.** A resource whose prerequisites the twin does not meet is demoted or excluded — recommendations match the learner's stage, not mere popularity. Introductory material is never gated.
- **Threshold-band decisions.** A strong assessment score completes the milestone and unlocks the next; a weak one inserts targeted remediation; high proficiency skips introductory material. Routine completions never mutate the plan — once formed, only hard evidence reshapes it — and exhausting all actionable material always opens the next milestone, so a path can never dead-end.
- **Progress is event-sourced.** The progress table is append-only; every summary, pace forecast and completion percentage is derived from it, so no displayed number can drift from the history behind it.
- **Career readiness, honestly.** A derived composite over technical skills, assessment performance, project work and momentum — a dimension with no evidence is reported *missing* and removed from the weighting, never counted as zero.

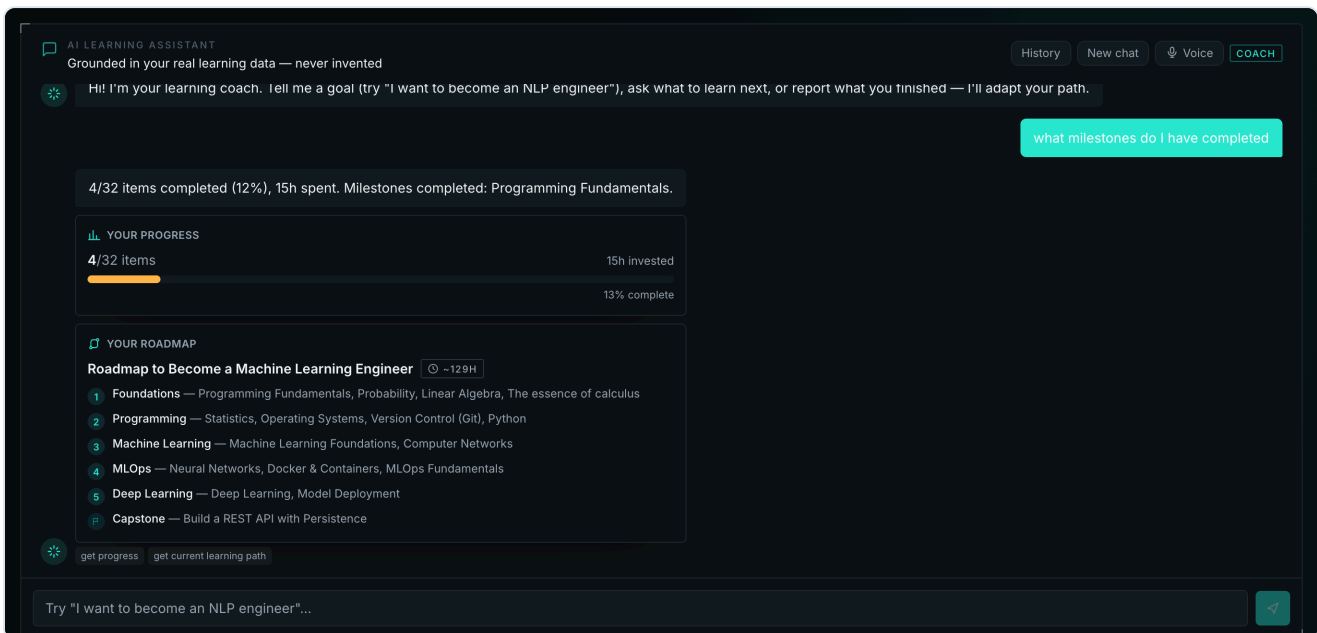


Overview: goal progress, pace forecast ("on plan - ~13.3w left") and career readiness — every figure derived from events, none stored.

06 The Voice AI Coach

A coach you can **talk to** — and one that can prove where every answer came from. Deterministic control flow around a generative step: the model never decides what to fetch, and every reply shows its receipts.

```
speak → speech-to-text ↙
type —————→ detect intent (rule-based, most-specific-first)
                    |→ select tools → run tools (real services)
                    |→ compose grounded reply → LLM rephrase (grounding-checked)
reply ← text-to-speech ↘ persist turn
```

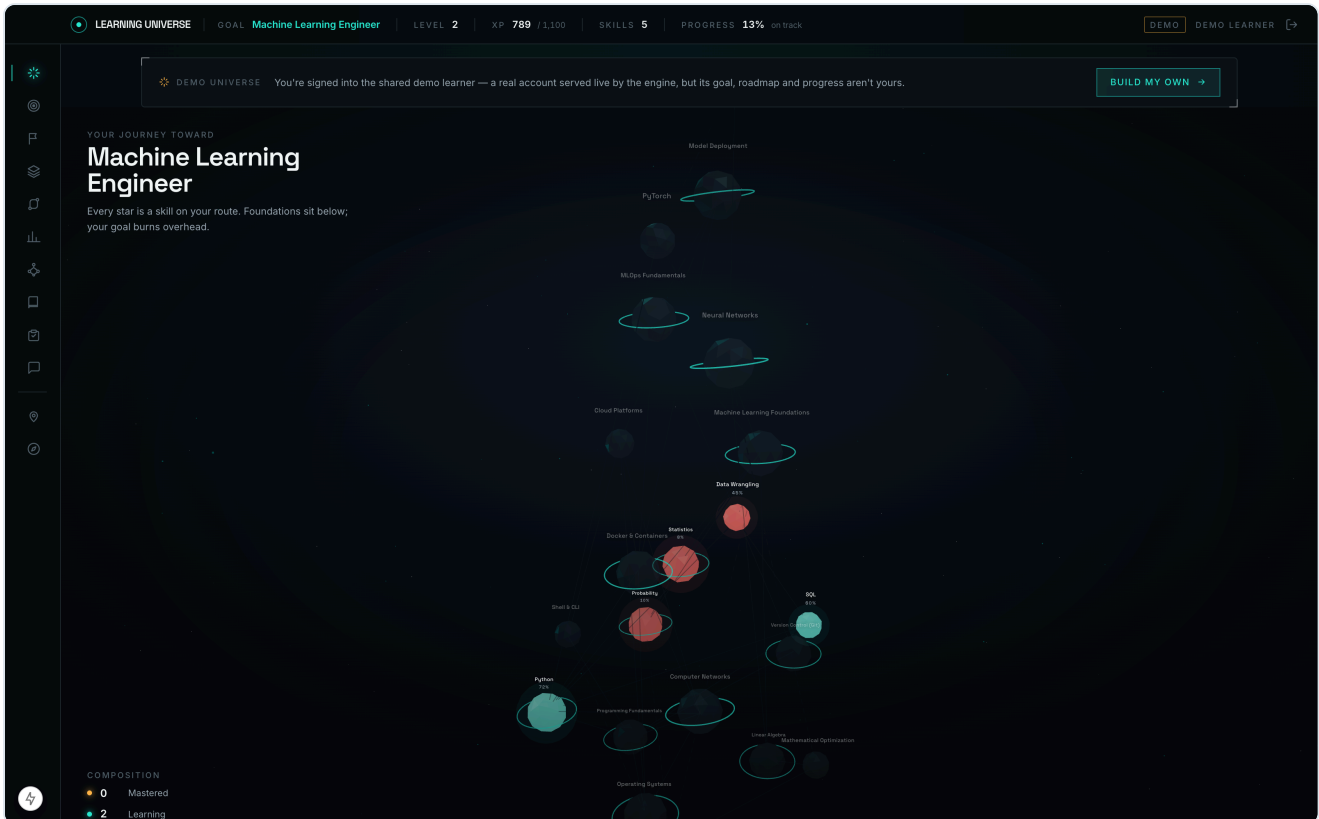


"What milestones do I have completed?" — answered from the learner's real path, with the tools that ran shown as chips beneath the reply, and the Voice control in the header.

- **One pipeline, two mouths.** A spoken turn calls exactly the same code as a typed one — intent engine, tools, grounding — so the two modes cannot drift apart. Speech runs on the browser's own Web Speech API: no key, no extra dependency.
- **Eleven tools, one truth.** Profile, gaps, path, progress, recommendations, next action, catalogue search, progress updates, goal prerequisites, skill relationships and career suggestions — every fact is re-fetched each turn. Conversation memory stores dialogue only; the coach cannot drift from stored state because it holds none.
- **A sentence can carry three things.** Each turn is also scanned for a time budget ("an hour a day" → 7 h/week) and claimed skills — feeding the twin under its evidence hierarchy.
- **Built for the ear.** Barge-in — speaking over a reply cancels it (and stops the microphone hearing the synthesiser); "85%" is spoken as "85 percent"; URLs become "the link on screen"; a browser without speech support keeps text chat and says so.
- **No unfulfillable offers.** The coach is forbidden from "would you like me to look that up?" theatre — tools have already run; when data is missing it says so and names the exact question to ask instead.

07 The 3D Learning Universe

The twin, the plan and the adaptation all render as one thing: a full-bleed, explorable galaxy that is **unique to each learner**.



The demo learner's universe: goal overhead, foundations below, mastery as colour, undiscovered skills dim in the knowledge fog.

- **Altitude is prerequisite depth** — foundations orbit at the bottom, the goal burns overhead; learning literally reads as ascending. Mastered skills burn amber, learning glows teal, weak pulses coral, and the undiscovered sits translucent in the **knowledge fog**.
- **Progression is game-like.** A HUD carries level, XP and pace — XP is *derived, never stored*, a pure function of the twin and the event log. Selecting a star eases the camera in and lights its full prerequisite route with travelling charge packets.
- **The coach drives the world.** When a reply names a skill, its star is selected and pulsed — "how comfortable are you with Linear Algebra?" physically points at Linear Algebra.
- **A projection, never a decoration.** The galaxy and the 2D knowledge graph render the same `GraphModel`; nothing appears that the engines and the learner's data did not put there. Even the landing page mounts this scene with the live demo learner's graph — its telemetry is fetched, never hardcoded.

08 Real Content & Trust Guarantees

- **Verified, not scraped-and-hoped.** The roadmap plans over real YouTube courses whose title, channel and runtime were read from YouTube's own metadata — every hour estimate is an actual runtime. In-app project briefs and checkpoints complete the mix.
- **Discovery with an agent, floored by an engine.** When a skill has no content, the pipeline searches; a pure selection engine filters the noise raw search returns (teasers, career-advice videos, wrong-language titles, one pick per channel); the model judges the best of that vetted shortlist — and can only choose among candidates the engine approved.
- **Self-healing.** A nightly health run re-checks every video; only a provider that can *prove* absence may deactivate a resource. Recovery is unrestricted in the other direction.

Trust & consistency guarantees

No model, no invented facts — ever.

Without a configured model, open subject questions are answered from the catalogue's stored course descriptions — curated facts — and the coach says plainly when that is all it has. A model adds fluency, never authority.

Reviewer grading replays the full pipeline.

Marking a short answer correct recovers the proficiency the submission blended in (the update is linear, so it inverts exactly), re-applies the reviewed score, and re-runs the adaptive decisions — a milestone earned by the corrected score unlocks then, not never.

Declared prior courses match carefully, then stop.

"CS50 Python" resolves to the Harvard course only when every identity token matches, exactly one candidate qualifies, and no stated provider disagrees — anything weaker shows the course again rather than hiding one the learner never took.

Feedback lands where it was given.

A thumbs-down demotes exactly that resource (latest signal wins); provider averages remain only as the prior for unrated material.

09 Technical Specification

LAYER	TECHNOLOGY
Backend	FastAPI · Pydantic v2 · SQLAlchemy 2.0 (async, asyncpg) · Alembic · PostgreSQL 16 + pgvector · Redis (shared query-embedding cache) · Docker Compose
Frontend	Next.js 15 (App Router) · React 19 · TypeScript · Tailwind · Three.js (react-three-fiber) · Recharts · Web Speech API
AI providers	LLM: <code>mock</code> <code>claude</code> <code>openai</code> · Embeddings: <code>mock</code> <code>openai</code> <code>sentence_transformer</code> · Catalogue: <code>none</code> <code>scrape</code> <code>youtube</code> — all settings-driven, never hard-coded, credentials validated lazily; the whole stack runs with no API key at all

Architecture

```

routers/      HTTP only – auth guard, serialisation, status codes
services/     business rules, transaction boundary, domain exceptions
repositories/ SQLAlchemy queries – flush, never commit
engines/      PURE decision logic – no DB, no clock, no network, no model
llm/ · embeddings/ · catalogue/  provider-agnostic seams with typed contracts

```

- **21 tables** across identity, the skill DAG (cycle-checked at write time), goals, catalogue, paths, assessments, event-sourced progress and conversations; pgvector columns power semantic retrieval.
- **~60 REST endpoints** under `/api/v1`: auth (JWT), profiles and the skill twin, graph queries, gap analysis, path generation and regeneration, per-item explanations, assessments with admin review, adaptive updates, progress, readiness, discovery, chat. Errors are RFC 7807; ownership reads as 404 so ids cannot be probed.
- **Ten dashboard sections** over one typed API client; the demo mode signs into a real seeded learner served by the live API, so there is no fake data anywhere in the product.
- **415 automated tests**. Engine tests run without any database — everything in `engines/` is pure. API tests run in-process against the real database. Each trust guarantee is pinned by dedicated tests.

Security & privacy posture

- JWT auth with role separation; catalogue writes and review queues are admin-only; every learner-scoped query filters by the authenticated user at the repository level.
- Secrets live in environment configuration outside images; the API refuses to boot in production with the development signing key; provider keys are never logged.
- Browser speech recognition streams audio to the browser vendor's service — the UI states this before the microphone is ever opened.

The universe is already running.

The full system — frontend, API, database, live AI providers — is deployed and publicly reachable:

LIVE APPLICATION

<https://pathwise-frontend-8qyh.onrender.com>

Explore without an account via Enter the Universe → the demo, or sign up and chart your own route — the landing page itself renders the live demo learner's galaxy straight from the production engine.

⌚ PLEASE NOTE — FREE-TIER HOSTING

The service is deployed on Render's **free tier**, which puts idle services to sleep. On first visit, **please allow up to 5 minutes** for the frontend and the backend to wake — the page may load before the galaxy and data appear. A single refresh after the wait brings everything to life; every visit after that is instant while the services stay warm.